

短学期学习指导

资料撰写：陶利民 2025.6.25

一、准备工作

1、安装配置软件列表

- **MySQL 8.0**

下载：<https://dev.mysql.com/downloads/mysql/>

执行版安装方法：https://blog.csdn.net/weixin_59131972/article/details/128056059

压缩包版安装方法：https://blog.csdn.net/weixin_65213208/article/details/126910055

- 数据库图形管理界面工具

Navicat for MySQL 或 SQLyog(免费使用)

- **Java 8/11/17**

下载：<http://www.codebaoku.com/jdk/jdk-index.html>

- **IntelliJ IDEA**

Java编程语言的集成开发环境

下载：<https://www.jetbrains.com.cn/idea/download/?section=windows>

IDEA配置好JDK：<https://www.quanxiaoha.com/idea/idea-set-jdk.html>

也可用：Visual Studio Code（简称VS Code）

<https://visualstudio.microsoft.com/zh-hans/>

- **MAVEN**

项目管理和构建自动化工具

下载安装：<https://blog.csdn.net/bakelff/article/details/123049992>

IDEA配置好MAVEN：https://blog.csdn.net/qg_51447496/article/details/128057628

- **git**

版本控制系统

git安装及IDEA配置：https://blog.csdn.net/geimogo_/article/details/130181618

2、注册账号

在gitee或github注册账号。

- **gitee**

<https://gitee.com/>

- github

<https://github.com/>

二、项目管理：MAVEN

为什么使用 Maven

1、管理JAR包

由于 Java 的生态非常丰富，无论你想实现什么功能，都能找到对应的工具类，这些工具类都是以 jar 包的形式出现的，例如 Spring, SpringMVC、MyBatis、数据库驱动，等等，都是以 jar 包的形式出现的，jar 包之间会有关联，在使用一个依赖之前，还需要确定这个依赖所依赖的其他依赖，所以，当项目比较大的时候，依赖管理会变得非常麻烦臃肿，这是 Maven 解决的第一个问题。

Maven 就是一款帮助程序员构建项目的工具，我们只需要告诉 Maven 需要哪些 jar 包，它会帮助我们下载所有的 Jar，极大提升开发效率。

2、项目打包

Maven 还可以处理多模块项目。简单的项目，单模块分包处理即可，如果项目比较复杂，要做成多模块项目，例如一个电商项目有订单模块、会员模块、商品模块、支付模块...，一般来说，多模块项目，每一个模块无法独立运行，要多个模块合在一起，项目才可以运行，这个时候，借助 Maven 工具，可以实现项目的一键打包。

Maven 是什么

Maven 是一个项目管理工具，它包含了一个**项目对象模型（Project Object Model）**，反映在配置中，就是一个 **pom.xml** 文件。是一组标准集合，一个项目的生命周期、一个依赖管理系统，另外还包括定义在项目生命周期阶段的插件(plugin)以及目标(goal)。

当我们使用 Maven 的使用，通过一个自定义的项目对象模型，pom.xml 来详细描述我们自己的项目。

Maven 中的有两大核心：

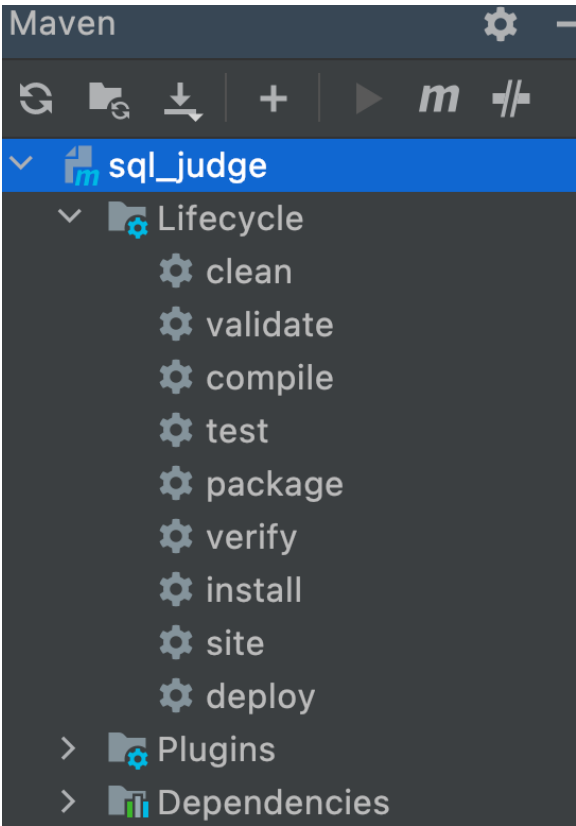
- **依赖管理**：对 jar 的统一管理(Maven 提供了一个 Maven 的中央仓库，<https://mvnrepository.com/>，当我们在项目中添加完依赖之后，Maven 会自动去中央仓库下载相关的依赖，并且解决依赖的依赖问题)。
- **项目构建**：对项目进行编译、测试、打包、部署、上传到私服等。

Maven 常用命令

Maven 中有一些常见的命令，如下表所示。

常用命令	中文含义	说明
mvn clean	清理	这个命令可以用来清理已经编译好的文件
mvn compile	编译	将 Java 代码编译成 Class 文件
mvn test	测试	项目测试
mvn package	打包	根据用户的配置，将项目打成 jar 包或者 war 包
mvn install	安装	手动向本地仓库安装一个 jar
mvn deploy	上传	将 jar 上传到私服

IDEA可使用菜单，不使用命令，如下图所示：



MAVEN项目案例——简单成绩管理系统

功能需求

一个简单成绩管理系统，有两类用户：管理员和学生。

- 登录功能
管理员凭工号和密码登录系统，学生凭学号和密码登录系统。两类用户登录系统后，其功能是不一样的。
- 管理员功能
管理学生信息：学生信息的增、删、改、查。
管理课程信息：课程信息的增、删、改、查；
成绩管理：录入成绩，修改成绩，查询成绩，删除成绩，统计成绩。

- 学生功能
查询选课成绩。

系统数据表结构

管理员用户表、学生表、课程表、选课表属于数据库 School（DBMS使用MySQL 8.0），其各自的数据结构如下：

用户表user：

序号	列名	含义	数据类型	长度	约束
1	emp_stu_no	工号/学号	字符型(char)	10	主码
2	name	姓名	字符型(char)	20	
3	phone	联系电话	字符型(char)	15	
4	password	登录密码	字符型(char)	10	
5	category	用户类型	整数(smallint)	10	1-管理员,2-学生

学生表student：

序号	列名	含义	数据类型	长度	约束
1	Sno	学号	字符型(char)	10	主码
2	Sname	姓名	字符型(varchar)	8	
3	Ssex	性别	字符型(char)	2	
4	Sage	年龄	整数 (smallint)		
5	sdept	系科	字符型(varchar)	15	

课程表 course：

序号	列名	含义	数据类型	长度	约束
1	Cno	课程号	字符型(char)	4	主码
2	cname	课程名	字符型(varchar)	20	
3	Cpno	先修课	字符型(char)	4	
4	Ccredit	学分	短整数 (tinyint)		

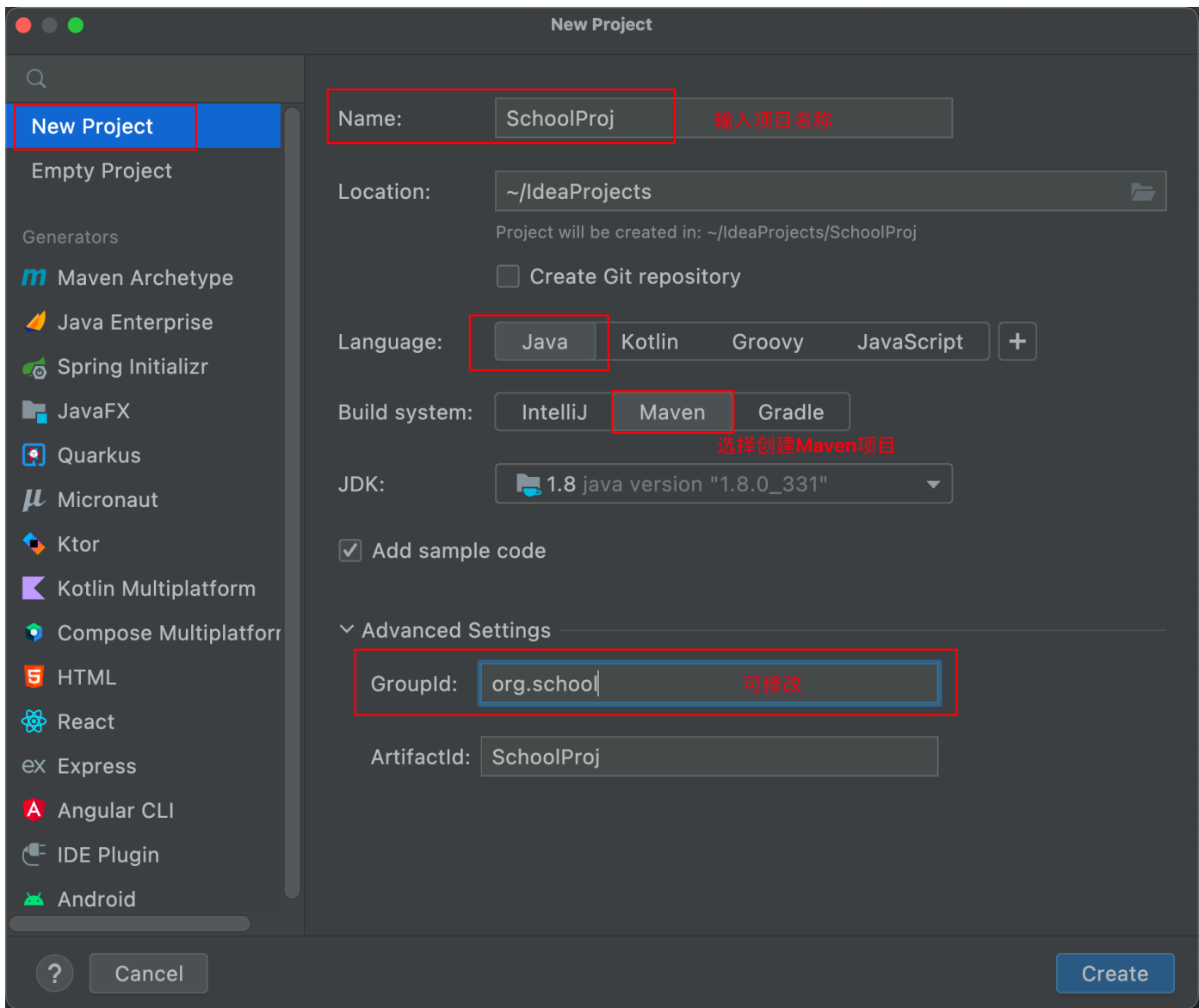
学生选课表sc：

序号	列名	含义	数据类型	长度	约束
1	Sno	学号	字符型(char)	10	主码，外码
2	Cno	课程号	字符型(char)	4	主码，外码
3	Grade	成绩	小数(decimal)	8,2	

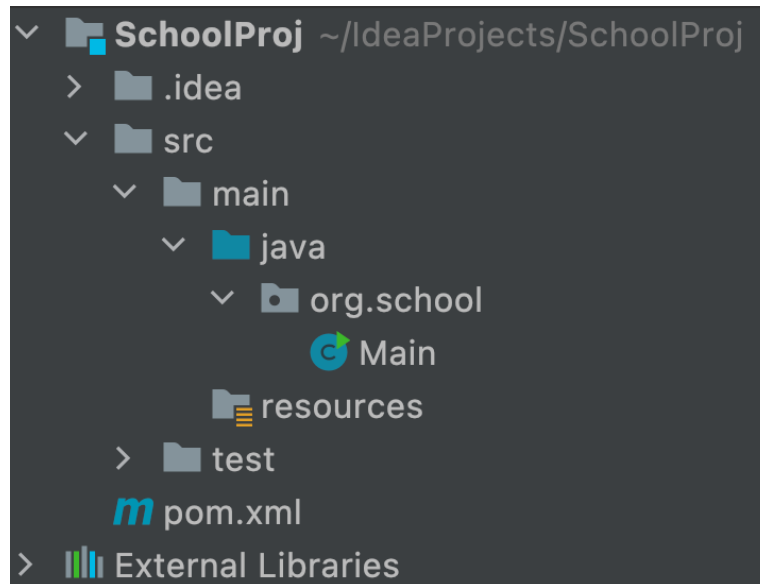
创建MAVEN项目

1、创建项目

创建一个JavaSE 的MAVEN项目，File->New->Project，如下图所示，这样就可以创建一个MAVEN项目了，项目名称为：ShoolProj。



创建好的项目结构如下图所示：



Main.java是程序执行的入口。

2、pom.xml

从上图可以看到有个pom.xml文件，pom.xml文件是Maven进行工作的主要配置文件，项目中要用到的jar包也是在此文件中引入的。

pom.xml的详细解析参考：

https://blog.csdn.net/weixin_42601136/article/details/121806064

<https://www.cnblogs.com/wkrbky/p/6353285.html>

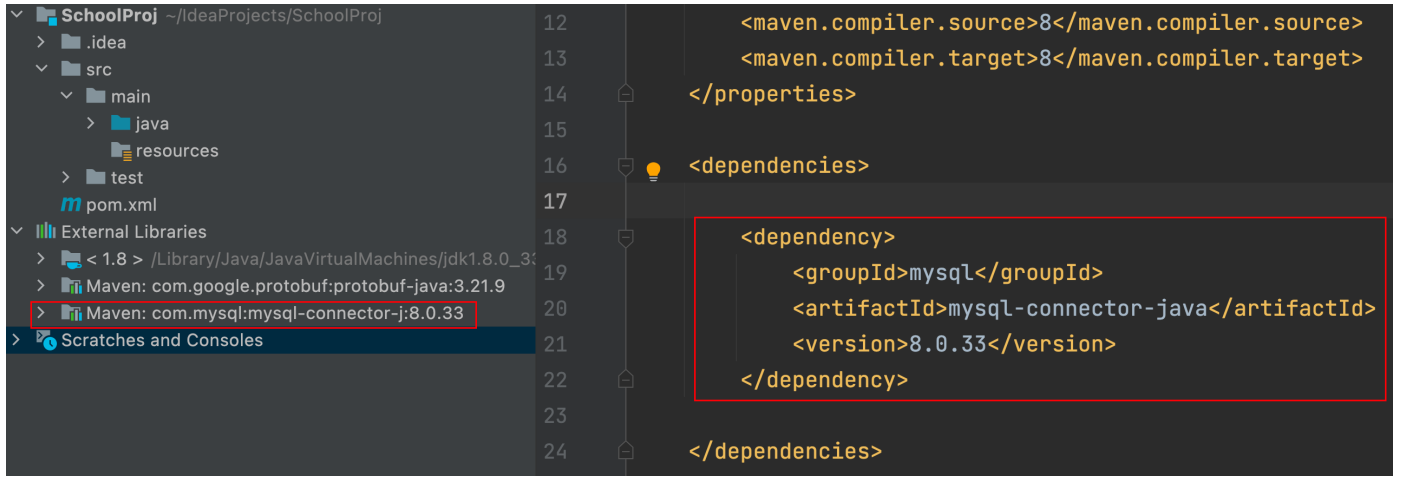
3、引入JAR包

在简单成绩管理系统中，Java去访问MySQL数据库，需要用到JDBC驱动mysql-connector-java-8.0.jar。使用Maven来管理项目，就可在pom.xml中配置依赖，这样就会自动从Maven的中央仓库（<https://mvnrepository.com/>）下载JAR包。当然，也可以自定义仓库。

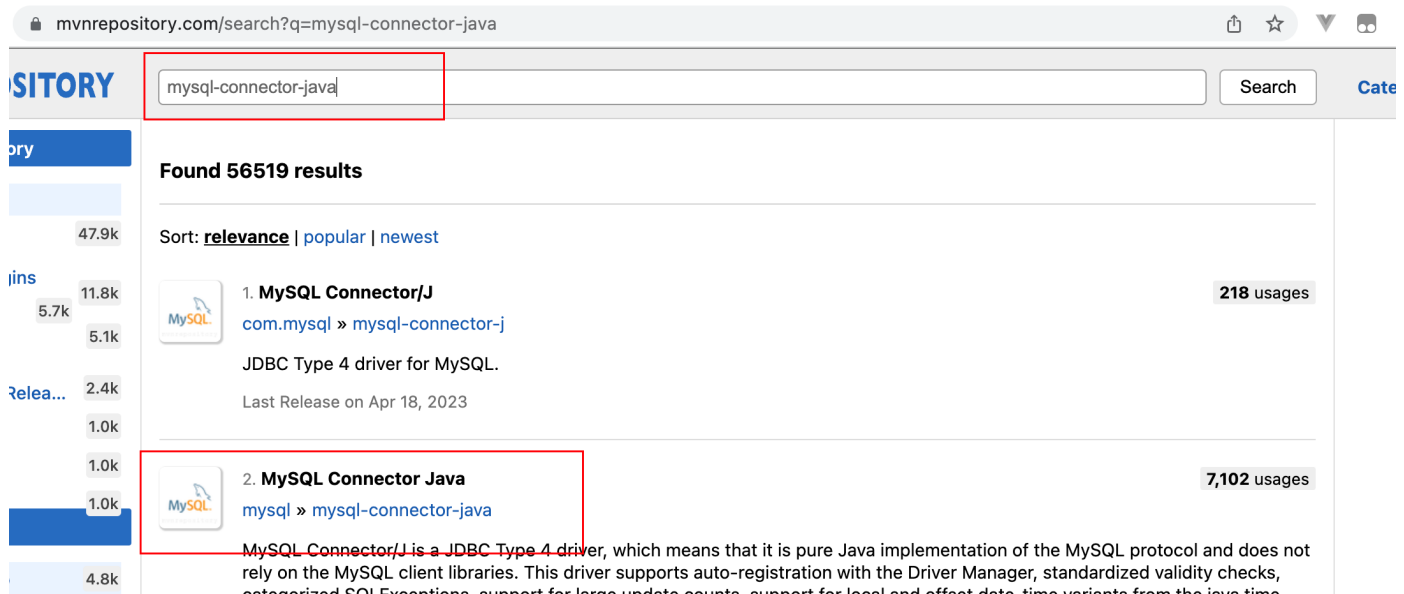
JDBC驱动JAR包的依赖代码如下：

```
1 <dependency>
2   <groupId>mysql</groupId>
3   <artifactId>mysql-connector-java</artifactId>
4   <version>8.0.33</version>
5 </dependency>
```

加上依赖，刷新Maven项目后，可看到左侧项目结构中的mysql-connector-java-8.0.33.jar。



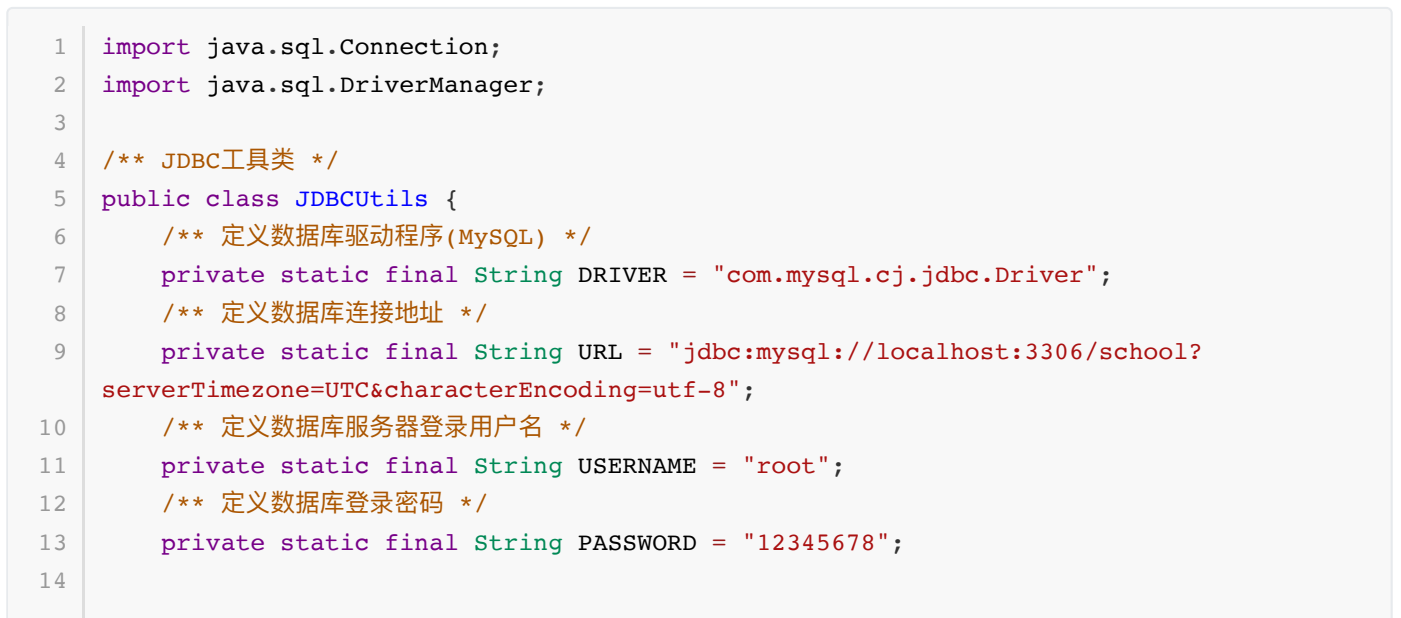
JAR包的依赖可去<https://mvnrepository.com/>查找，如下图所示。



4、代码示例

JDBC驱动已导入，现在就可以用Java去访问MySQL数据库了。

JDBCUtils.java



```

15      /**
16       * 利用静态初始化块加载数据库驱动程序 静态初始化块在类加载时就调用,且只调用一次
17       */
18      static {
19          try {
20              // 加载数据库驱动程序
21              Class.forName(DRIVER);
22              System.out.println("数据库驱动程序加载成功! ");
23          } catch (Exception e) {
24              System.out.println("数据库驱动程序加载失败: " + e.getMessage());
25          }
26      }
27
28      /** 获取Connection对象的方法 */
29      public static Connection getConnection() {
30          Connection conn = null;
31          try {
32              // 连接数据库, 获取Connection对象
33              conn = DriverManager.getConnection(URL, USERNAME, PASSWORD);
34              System.out.println("连接数据库成功! ");
35          } catch (Exception e) {
36              e.printStackTrace();
37              System.out.println("连接数据库失败: " + e.getMessage());
38          }
39          return conn;
40      }
41  }

```

App.java

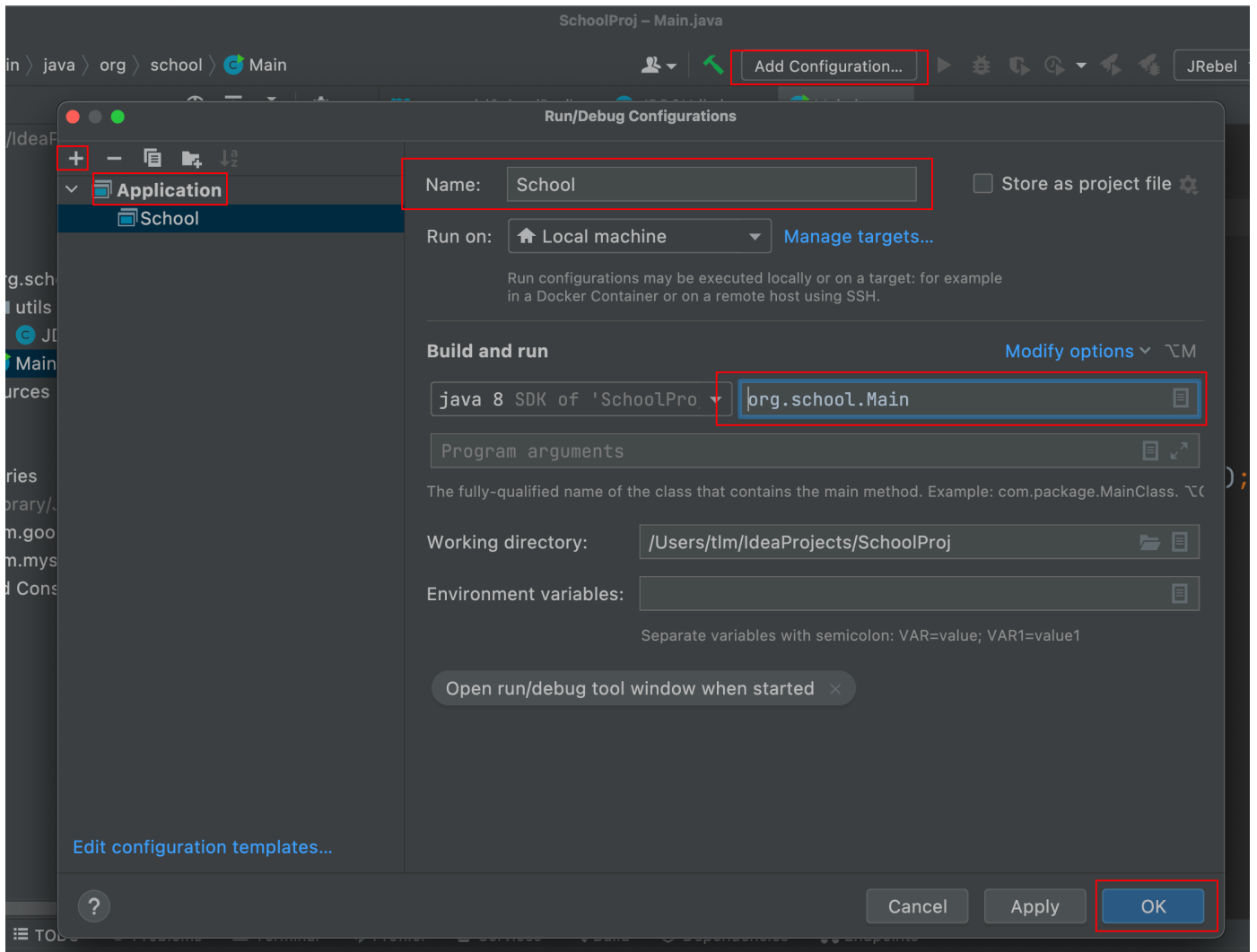
```

1  public class App
2  {
3      public static void main( String[] args )
4      {
5          // 测试驱动加载、数据库连接
6          Connection connection= JDBCUtils.getConnection();
7      }
8  }

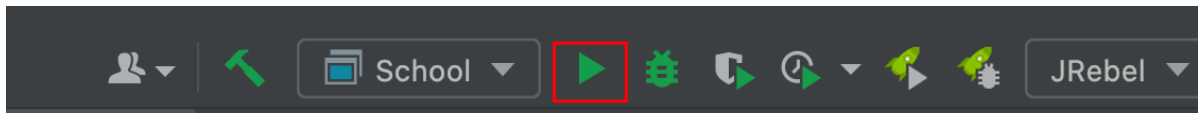
```

5、运行项目

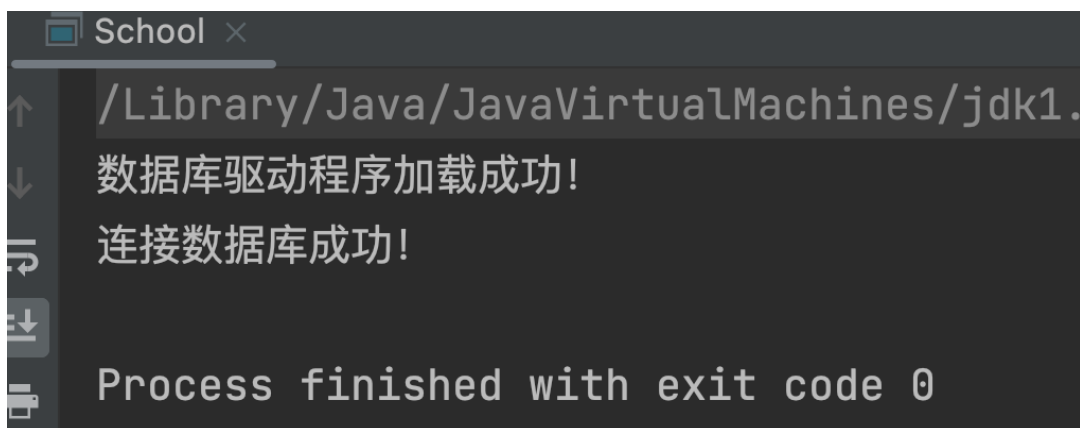
要运行项目，还需配置一下，如下图所示。



配置好就可以点击绿色三角形按钮运行了。

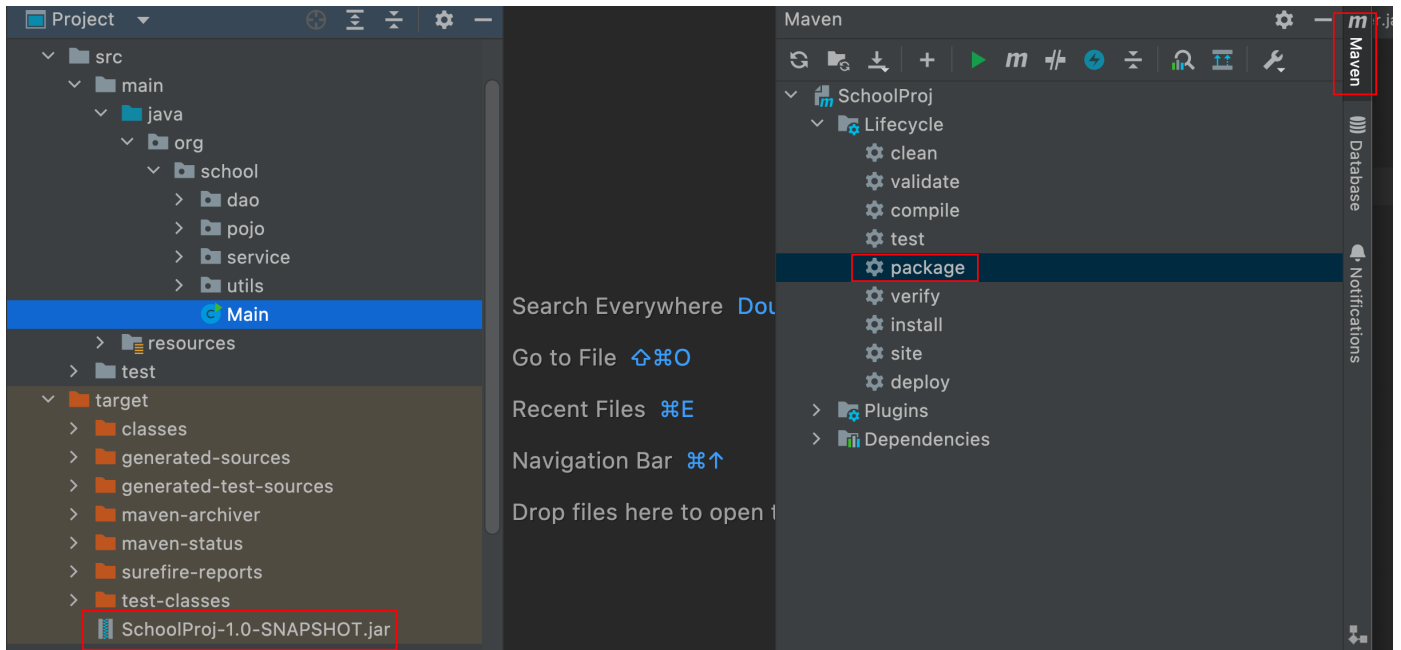


运行结果如下：



6、项目打包

可利用右侧Maven工具进行打包，如下图所示，双击package就可进行打包。打包成功后，在左侧target下可看到jar包。



运行JAR包：

打包好的JAR包可通过命令直接运行，命令如下：

```
java -jar SchoolProj-1.0-SNAPSHOT.jar
```

三、MyBatis

在Maven项目的pom.xml中配置好JDBC驱动JAR包的依赖代码，可以使用JDBC驱动了。不过，直接使用JDBC进行项目开发，效率不高，便可以使用据持久化框架，如MyBatis、Hibernate等。

MyBatis简介

MyBatis 是一款优秀的持久层框架，它支持自定义 SQL、存储过程以及高级映射。MyBatis 免除了几乎所有的 JDBC 代码以及设置参数和获取结果集的工作。MyBatis 可以通过简单的 XML 或注解来配置和映射原始类型、接口和 Java POJO（Plain Old Java Objects，普通老式 Java 对象）为数据库中的记录。

MyBatis 是一个开源、轻量级的数据持久化框架，是 JDBC 和 Hibernate 的替代方案。MyBatis 内部封装了 JDBC，简化了加载驱动、创建连接、创建 statement 等繁杂的过程，开发者只需要关注 SQL 语句本身。

数据持久化是将内存中的数据模型转换为存储模型，以及将存储模型转换为内存中数据模型的统称。例如，文件的存储、数据的读取以及对数据表的增删改查等都是数据持久化操作。

MyBatis 官方文档：<https://mybatis.org/mybatis-3/zh/>

Mybatis的作用

- Mybatis就是帮助程序员将数据存取到数据库里面。
- 传统的jdbc操作，有很多重复代码块。比如：数据取出时的封装，数据库的建立连接等等...，通过框架可以减少重复代码,提高开发效率。
- MyBatis 是一个半自动化的ORM框架 (Object Relationship Mapping) -->对象关系映射
- 所有的事情，不用Mybatis依旧可以做到，只是用了它，会更加方便更加简单，开发更快速。

MyBatis的优点

- 简单易学：本身就很小且简单。没有任何第三方依赖，最简单安装只要两个jar文件+配置几个sql映射文件就可以了，易于学习，易于使用，通过文档和源代码，可以比较完全的掌握它的设计思路和实现。
- 灵活：Mybatis不会对应应用程序或者数据库的现有设计强加任何影响。**sql写在xml里**，便于统一管理和优化。通过sql语句可以满足操作数据库的所有需求。
- 解除sql与程序代码的耦合：通过提供DAO层，将业务逻辑和数据访问逻辑分离，使系统的设计更清晰，更易维护，更易单元测试。sql和代码的分离，提高了可维护性。
- 提供xml标签，支持编写动态sql。
- 现在主流使用方法

MyBatis基本使用方法

继续完善项目ShoolProj，在此项目中使用Mybatis。

1、导入MyBatis JAR包

在pom.xml文件中导入MyBatis相关jar包：

```
1      <dependency>
2          <groupId>mysql</groupId>
3          <artifactId>mysql-connector-java</artifactId>
4          <version>8.0.33</version>
5      </dependency>
6      <dependency>
7          <groupId>org.mybatis</groupId>
8          <artifactId>mybatis</artifactId>
9          <version>3.5.13</version>
10     </dependency>
11     <!--JUnit：单元测试框架-->
12     <dependency>
13         <groupId>junit</groupId>
14         <artifactId>junit</artifactId>
15         <version>4.13.2</version>
16     </dependency>
17
18     <build>
19         <!--这个元素描述了项目相关的所有资源路径列表，例如和项目相关的属性文件，这些资源被包含在最终
20         的打包文件里。-->
21         <resources>
22             <resource>
23                 <directory>src/main/java</directory>
24                 <includes>
25                     <include>**/*.properties</include>
26                     <include>**/*.xml</include>
27                 </includes>
28                 <filtering>>false</filtering>
29             </resource>
30             <resource>
31                 <!-- 描述存放资源的目录，该路径相对POM路径 -->
32                 <directory>src/main/resources</directory>
33                 <includes>
34                     <include>**/*.properties</include>
```

```

34         <include>**/*.xml</include>
35     </includes>
36     <filtering>>false</filtering>
37 </resource>
38 </resources>
39 </build>
40

```

说明：

- JUnit是一个Java语言的单元测试框架。

参考：<https://zhuanlan.zhihu.com/p/344841036>

- 在节点中配置项目相关的所有资源路径。

参考：https://blog.51cto.com/u_5650011/5386924

2、编写Mybatis核心配置文件

编写Mybatis核心配置文件，在resources下创建mybatis-config.xml文件，配置自己的数据库地址、名字、密码以及mysql驱动。

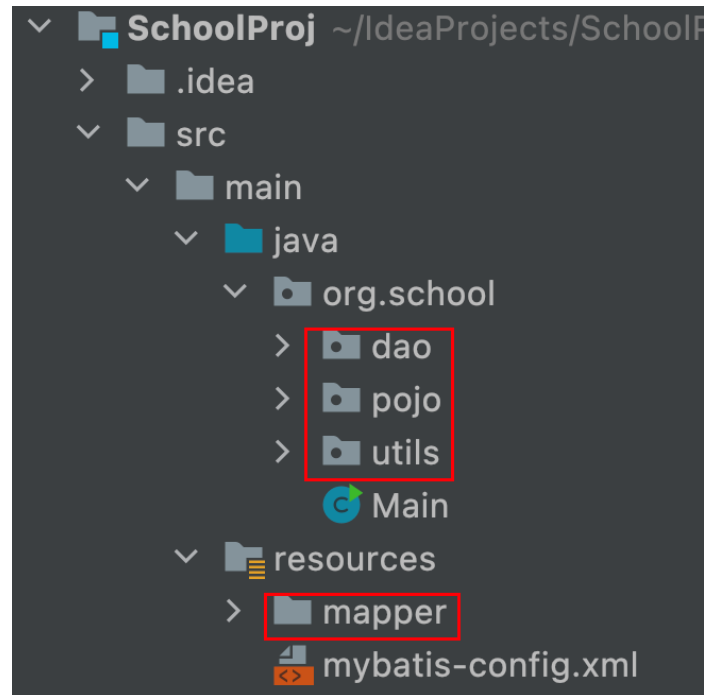
```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE configuration
3      PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
4      "http://mybatis.org/dtd/mybatis-3-config.dtd">
5  <!--configuration核心配置文件-->
6  <configuration>
7      <environments default="development">
8          <environment id="development">
9              <transactionManager type="JDBC" />
10             <dataSource type="POOLED">
11                 <property name="driver" value="com.mysql.cj.jdbc.Driver" />
12                 <property name="url" value="jdbc:mysql://localhost:3306/school?
useSSL=true&sueUnicode=true&characterEncoding=UTF-
8&serverTimezone=Asia/Shanghai" />
13                 <property name="username" value="root" />
14                 <property name="password" value="12345678" />
15             </dataSource>
16         </environment>
17     </environments>
18 </configuration>

```

3、划分工程目录结构

将工程目录按照层划分，如下图所示。



dao: DAO层负责访问数据库进行数据的操作，取得结果集，之后将结果集中的数据取出封装到VO类对象之后返回给service层（service后面会出现）。

pojo: plain ordinary java object，简单java对象；一般专指只有 setter/getter/toString 的简单类；pojo用于数据的临时传递，它只能装载数据，作为数据存储的载体，而不具有业务逻辑处理的能力。pojo层命名为entity也可，表示实体层。

utils: 工具类包，通用的且与项目业务无关的类的组合。

mapper: 存放Mybatis的Mapper映射文件。

4、编写MyBatis工具类

在utils包下新建->MybatisUtils.java文件,编写代码如下:

```
1  import org.apache.ibatis.io.Resources;
2  import org.apache.ibatis.session.SqlSession;
3  import org.apache.ibatis.session.SqlSessionFactory;
4  import org.apache.ibatis.session.SqlSessionFactoryBuilder;
5  import java.io.IOException;
6  import java.io.InputStream;
7
8  // sqlSessionFacotry -->sqlSession
9  public class MybatisUtils {
10     private static SqlSessionFactory sqlSessionFactory;
11     // 使用mybatis第一步，获取sqlSessionFactory对象
12     static {
13         try {
14             String resource = "mybatis-config.xml";
15             // Resources类：从类路径中加载资源
16             InputStream inputStream = Resources.getResourceAsStream(resource);
17             sqlSessionFactory = new SqlSessionFactoryBuilder().build(inputStream);
```

```

18         } catch (IOException e) {
19             e.printStackTrace();
20         }
21     }
22
23     // 既然有了sqlSessionFactory,顾名思义,我们就可以从中获得sqlSession的实例了
24     // sqlSession 完全包含了面向数据库执行SQL命令所需的所有方法
25     public static SqlSession getSqlSession(){
26         // openSession()方法的参数值设置成true时,可以开启事务自动提交功能。
27         return sqlSessionFactory.openSession(true);
28     }
29 }

```

说明:

SqlSessionFactory

- SqlSessionFactory是MyBatis的关键对象,它是个单个数据库映射关系经过编译后的内存镜像。
- SqlSessionFactory对象的实例可以通过SqlSessionFactoryBuilder对象类获得,而SqlSessionFactoryBuilder则可以从XML配置文件或一个预先定制的Configuration的实例构建出SqlSessionFactory的实例。
- 每一个MyBatis的应用程序都以一个SqlSessionFactory对象的实例为核心。
- SqlSessionFactory是线程安全的,SqlSessionFactory一旦被创建,应该在应用执行期间都存在。在应用运行期间不要重复创建多次,建议使用单例模式。
- SqlSessionFactory是创建SqlSession的工厂。

SqlSession

- SqlSession是MyBatis的关键对象,是执行持久化操作的独享,类似于JDBC中的Connection。
- 它是应用程序与持久层之间执行交互操作的一个单线程对象,也是MyBatis执行持久化操作的关键对象。
- SqlSession对象完全包含以数据库为背景的所有执行SQL操作的方法,它的底层封装了JDBC连接,可以用SqlSession实例来直接执行被映射的SQL语句。
- 每个线程都应该有它自己的SqlSession实例。
- SqlSession的实例不能被共享,同时SqlSession也是线程不安全的,绝对不能讲SqlSeesion实例的引用放在一个类的静态字段甚至是实例字段中。也绝不能将SqlSession实例的引用放在任何类型的管理范围中,比如Servlet当中的HttpSession对象中。
- 使用完SqlSeesion之后关闭Session很重要,应该确保使用finally块来关闭它。

Resources

- Resources类: 从类路径中加载资源。
- Resources可加载的配置文件

5、创建实体类

实体类存放数据库中对象信息。一般,数据库中每张数据表对应一个实体类,类名就是表名,实体类的属性对应数据表的字段。

在pojo包下创建Student.java文件,编写代码如下:

```

1  /** 学生类 */
2  public class Student {

```

```
3      // 学号
4      private String sno;
5      // 姓名
6      private String sname;
7      // 性别
8      private String ssex;
9      // 年龄
10     private int sage;
11     // 系科
12     private String sdept;
13
14     public String getSno() {
15         return sno;
16     }
17
18     public void setSno(String sno) {
19         this.sno = sno;
20     }
21
22     public String getSname() {
23         return sname;
24     }
25
26     public void setSname(String sname) {
27         this.sname = sname;
28     }
29
30     public String getSsex() {
31         return ssex;
32     }
33
34     public void setSsex(String ssex) {
35         this.ssex = ssex;
36     }
37
38     public int getSage() {
39         return sage;
40     }
41
42     public void setSage(int sage) {
43         this.sage = sage;
44     }
45
46     public String getSdept() {
47         return sdept;
48     }
49
50     public void setSdept(String sdept) {
51         this.sdept = sdept;
52     }
53 }
```

说明：一般情况，实体类成员变量的数据类型使用包装类型，这样可取值为null。

6、编写Mapper接口类

创建Mapper代理接口（dao接口），接口方法在映射文件中有关联。dao接口存放数据库操作。

在dao包下创建 StudentDao.interface文件,编写代码如下：

```
1  import org.school.pojo.Student;
2  import java.util.List;
3
4  public interface StudentDao {
5      //查询所有学生信息
6      List<Student> findAllStudent();
7
8      // 按学号查询学生信息
9      Student findStudentBySno(String sno);
10
11     // 根据输入的学生信息进行动态条件检索
12     List<Student> findStudent(Student student);
13
14     //增加一个学生信息
15     int addStudent(Student student);
16
17     //更改学生信息
18     int updateStudent(Student student);
19
20     //删除学生信息
21     int deleteStudentBySno(String sno);
22 }
```

7、编写Mapper.xml配置文件

mapper.xml是映射文件，用来存储实际的SQL语句。

在文件夹resources/mapper下创建 StudentMapper.xml文件,编写代码如下：

```
1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE mapper
3      PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4      "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5  <!--namespace=绑定一个对应的Dao/Mapper接口-->
6  <mapper namespace="org.school.dao.StudentDao">
7
8      <select id="findAllStudent" resultType="org.school.pojo.Student">
9          select * from student
10     </select>
11
12     <select id="findStudent" resultType="org.school.pojo.Student"
13         parameterType="org.school.pojo.Student">
14         select *
```



```

14         from student
15         <where>
16             <if test="sno != null and sno!=''">
17                 sno=#{sno}
18             </if>
19             <if test="sname != null and sname !='">
20                 and sname like concat('%', #{sname}, '%')
21             </if>
22             <if test="ssex != null and ssex!=''">
23                 and ssex=#{ssex}
24             </if>
25             <if test="sdept != null and sdept!=''">
26                 and sdept=#{sdept}
27             </if>
28         </where>
29     </select>
30
31     <select id="findStudentBySno" resultType="org.school.pojo.Student">
32         select * from student where sno=#{sno}
33     </select>
34
35     <!-- 对应的插入字段的名称 -->
36     <sql id="key">
37         <trim suffixOverrides=",">
38             <if test="sno!=null and sno!=''">
39                 sno,
40             </if>
41             <if test="sname!=null and sname!=''">
42                 sname,
43             </if>
44             <if test="ssex!=null and ssex!=''">
45                 ssex,
46             </if>
47             <if test="sage!=null and sage!=''">
48                 sage,
49             </if>
50             <if test="sdept!=null and sdept!=''">
51                 sdept,
52             </if>
53         </trim>
54     </sql>
55     <!-- 对应的插入字段的值 -->
56     <sql id="values">
57         <trim suffixOverrides=",">
58             <if test="sno!=null and sno!=''">
59                 #{sno},
60             </if>
61             <if test="sname!=null and sname!=''">
62                 #{sname},
63             </if>
64             <if test="ssex!=null and ssex!=''">
65                 #{ssex},

```

```

66         </if>
67         <if test="sage!=null and sage!=''">
68             #{sage},
69         </if>
70         <if test="sdept!=null and sdept!=''">
71             #{sdept},
72         </if>
73     </trim>
74 </sql>
75 <insert id="addStudent">
76     insert into student(<include refid="key"/>)
77     values(<include refid="values"/>)
78 </insert>
79
80 <update id="updateStudent">
81     update student
82     <set>
83         <if test="sno != null">sno=#{sno},</if>
84         <if test="sname != null">sname=#{sname},</if>
85         <if test="ssex != null">ssex=#{ssex},</if>
86         <if test="sage != null">sage=#{sage}</if>
87         <if test="sdept != null">sdept=#{sdept}</if>
88     </set>
89     where sno=#{sno}
90 </update>
91
92 <delete id="deleteStudentBySno">
93     delete from student where sno=#{sno}
94 </delete>
95 </mapper>

```

说明：

- **namespace="org.school.dao.StudentDao"**，就是第6步创建的Mapper代理接口。
- **id="findAllStudent"**，对应的Mapper代理接口方法的方法名。
- **id="findStudent"**，使用了动态SQL，这样就可以根据实体student的值动态组合查询条件。
- **id="addStudent"**，使用了动态SQL，这样可视情况，只新增实体部分属性的值。
- **id="updateStudent"**，使用了动态SQL，这样可视情况，只更新实体部分属性的值。
- 动态SQL，参考：

<https://zhuanlan.zhihu.com/p/435270797>

<https://zhuanlan.zhihu.com/p/360448887>

8、将Mapper.xml注册到mybatis-config.xml文件中

此时记得要将刚才的UserMapper.xml注册到mybatis-config.xml文件中，非常重要，此后每编写一个Mapper.xml文件都要注册一次！！！，在mybatis-config.xml文件更新为如下代码：

```

1 <?xml version="1.0" encoding="UTF-8" ?>

```

```

2  <!DOCTYPE configuration
3      PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
4      "http://mybatis.org/dtd/mybatis-3-config.dtd">
5  <!--configuration核心配置文件-->
6  <configuration>
7      <environments default="development">
8          <environment id="development">
9              <transactionManager type="JDBC"/>
10             <dataSource type="POOLED">
11                 <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
12                 <property name="url" value="jdbc:mysql://localhost:3306/school?
useSSL=true&sueUnicode=true&characterEncoding=UTF-
8&serverTimezone=Asia/Shanghai"/>
13                 <property name="username" value="root"/>
14                 <property name="password" value="12345678"/>
15             </dataSource>
16         </environment>
17     </environments>
18
19     <!--每一个Mapper.XML都需要在Mybatis核心配置文件中注册! -->
20     <mappers>
21         <mapper resource="mapper/StudentMapper.xml"/>
22     </mappers>
23 </configuration>

```

9、编写测试类，测试代码

在test->java->org->school->dao下创建UserDaoTest.java文件，编写代码如下：

```

1  import org.apache.ibatis.session.SqlSession;
2  import org.junit.Test;
3  import org.school.pojo.Student;
4  import org.school.utils.MybatisUtils;
5  import java.util.List;
6
7  public class UserDaoTest {
8      //第一步：获得sqlSession对象
9      private SqlSession sqlSession;
10     @Test
11     public void test(){
12         try {
13             // 获取sqlSession
14             sqlSession = MybatisUtils.getSqlSession();
15             // 获取mapper接口
16             StudentDao mapper = sqlSession.getMapper(StudentDao.class);
17
18             // 查询
19             Student student=mapper.findStudentBySno("200515003");
20             System.out.println(student);
21
22             Student stu=new Student();

```

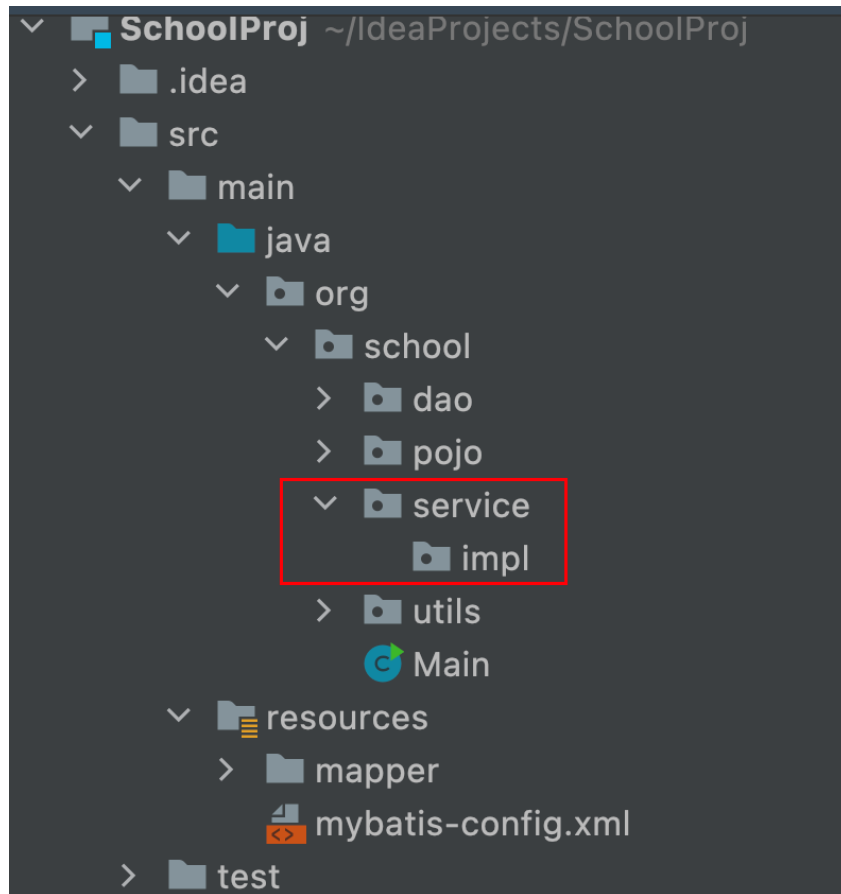
```

23         stu.setSno("200515003");
24         //stu.setName("张");
25         //stu.setSsex("女");
26         List<Student> studentList0=mapper.findStudent(stu);
27         for (Student student0 : studentList0) {
28             System.out.println(student0);
29         }
30
31         // 新增
32         Student stu1=new Student();
33         stu1.setSno("20020908");
34         stu1.setName("李飞飞");
35         // mapper.addStudent(stu1);
36
37         // 修改
38         stu1.setName("李明");
39         mapper.updateStudent(stu1);
40
41         // 删除
42         mapper.deleteStudentBySno("20020908");
43
44         // 查询全部学生
45         List<Student> studentList = mapper.findAllStudent();
46         for (Student student1 : studentList) {
47             System.out.println(student1);
48         }
49
50     }catch (Exception e){
51         e.printStackTrace();
52     }finally {
53         //关闭sqlSession
54         sqlSession.close();
55     }
56 }
57 }

```

10、进一步完善项目结构

至此，已介绍完MyBatis的基本使用方法。不过，项目开发实践中，在进行业务处理时，一般不会直接调用DAO层的接口，而时将业务逻辑处理封装到service层，在service层调用已定义的DAO层的接口。完善后的项目结构如下图所示，增加了一个service层；service层定义有接口及接口的实现类，接口的实现类放在impl包下。



service层：主要去负责一些业务逻辑处理。Service层的业务实现，具体要调用到已定义的DAO层的接口。封装Service层的业务逻辑有利于通用的业务逻辑的独立性和重复利用性。

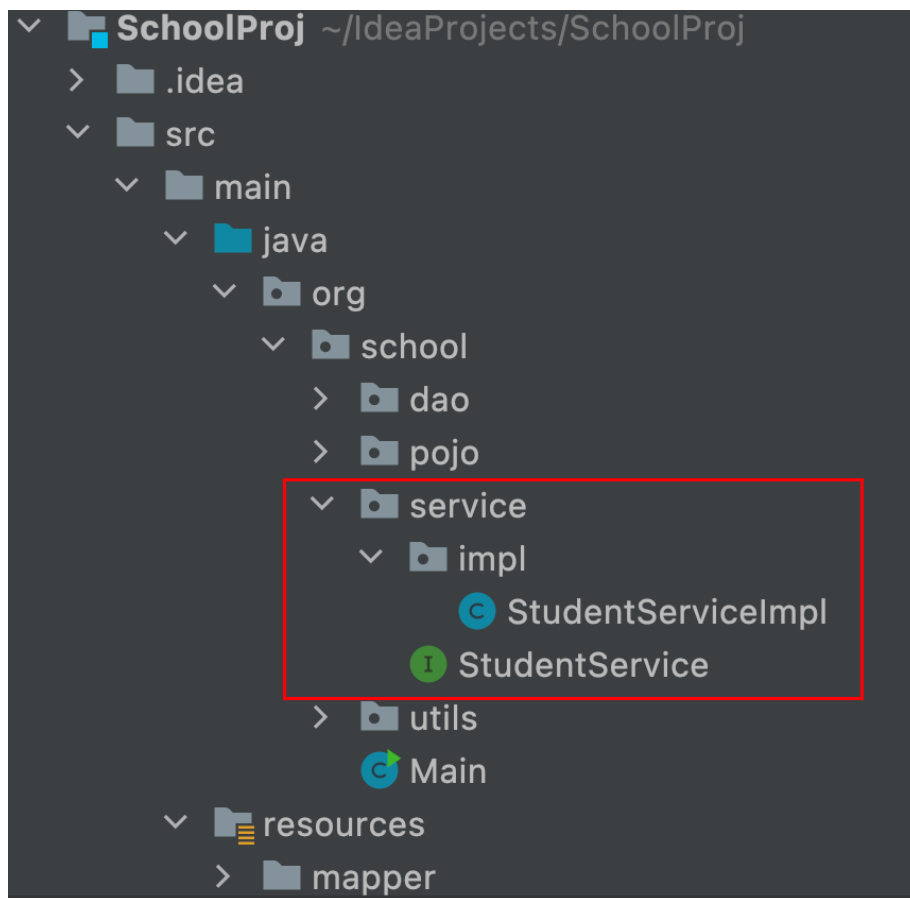
Step1：在service包下创建接口类StudentService，代码如下：

```
1  import org.school.pojo.Student;
2  import java.util.List;
3
4  public interface StudentService {
5      //查询所有学生信息
6      List<Student> findAllStudent();
7
8      // 按学号查询学生信息
9      Student findStudentBySno(String sno);
10
11     // 根据输入的学生信息进行动态条件检索
12     List<Student> findStudent(Student student);
13
14     //增加一个学生信息
15     int addStudent(Student student);
16
17     //更改学生信息
18     int updateStudent(Student student);
19
20     //删除学生信息
21     int deleteStudentBySno(String sno);
```

Step2: 在service.impl包下创建StudentService接口的实现类StudentServiceImpl，代码如下：

```
1  import org.apache.ibatis.session.SqlSession;
2  import org.school.dao.StudentDao;
3  import org.school.pojo.Student;
4  import org.school.service.StudentService;
5  import org.school.utils.MybatisUtils;
6  import java.util.List;
7
8  public class StudentServiceImpl implements StudentService {
9      private SqlSession sqlSession;
10     private StudentDao mapper;
11
12     public StudentServiceImpl(){
13         // 获取sqlSession
14         sqlSession = MybatisUtils.getSqlSession();
15         // 获取mapper接口
16         mapper = sqlSession.getMapper(StudentDao.class);
17     }
18
19     @Override
20     public List<Student> findAllStudent() {
21         return mapper.findAllStudent();
22     }
23
24     @Override
25     public Student findStudentBySno(String sno) {
26         return mapper.findStudentBySno(sno);
27     }
28
29     @Override
30     public List<Student> findStudent(Student student) {
31         return mapper.findStudent(student);
32     }
33
34     @Override
35     public int addStudent(Student student) {
36         return mapper.addStudent(student);
37     }
38
39     @Override
40     public int updateStudent(Student student) {
41         return mapper.updateStudent(student);
42     }
43
44     @Override
45     public int deleteStudentBySno(String sno) {
46         return mapper.deleteStudentBySno(sno);
47     }
48 }
```

最终，service层的结构如下：



Step3: 在main方法中测试，代码如下：

```
1  import org.school.pojo.Student;
2  import org.school.service.StudentService;
3  import org.school.service.impl.StudentServiceImpl;
4  import org.school.utils.JDBCUtils;
5  import java.sql.Connection;
6  import java.util.List;
7
8  public class Main {
9      public static void main(String[] args) {
10         // 测试驱动加载、数据库连接
11         // Connection connection= JDBCUtils.getConnection();
12
13         // 测试service
14         StudentService studentService=new StudentServiceImpl();
15         List<Student> studentList=studentService.findAllStudent();
16         for(Student student:studentList){
17             System.out.println(student);
18         }
19     }
20 }
21 }
```

Mybatis使用注解开发

Mybatis最初配置信息是基于 XML ,映射语句(SQL)也是定义在 XML 中的。而到MyBatis 3提供了新的基于注解的配置。

使用注解开发就是无需再配置Mapper.xml文件，直接在Mapper接口中利用注解实现SQL语句。

示例：

```
1 @Select("select * from student")
2 List<Student> findAllStudent();
```

进一步学习，参考以下文献：

<https://www.cnblogs.com/jmsstudy/archive/2022/09/14/16694954.html>

<https://www.cnblogs.com/kohler21/p/17016992.html>

https://blog.csdn.net/m0_46979453/article/details/122022095

Mybatis代码生成器

MyBatis Generator 是 MyBatis 提供的一个**代码自动生成工具**。能够通过简单的配置去帮我们生成数据表所对应的 PO、DAO、XML 等文件，减去我们手动去生成这些文件的时间，有效提高开发效率。

MyBatis Generator 是一个独立工具，你可以下载它的jar包来运行、也可以在 Ant 和 maven 运行。

具体使用，参考以下文献：

IDEA使用mybatis-generator

<https://www.jianshu.com/p/b519e9ef605f>

在IDEA中使用MyBatis Generator

https://blog.csdn.net/m0_51152782/article/details/120410790

Mybatis generator 生成 Mapper 方法不全

https://blog.csdn.net/weixin_43941364/article/details/109065010

【MyBtis】 Mybatis generator 生成 Mapper 方法不全

https://blog.csdn.net/weixin_44707283/article/details/122258457

MyBatis逆向工程generatorConfig配置文件的Table中generatedKey的作用

<https://blog.csdn.net/u013282737/article/details/89491187>

参考文献

MyBatis 官方文档

<https://mybatis.org/mybatis-3/zh/>

MyBatis教程（看这一篇就够了）

<https://blog.csdn.net/lsl5713/article/details/124451398>

MyBatis的进一步学习，要参考以下文献：

IDEA——手把手教你mybatis的使用(新手教程)

https://blog.csdn.net/weixin_46521008/article/details/126733691

idea中mybatis配置（解决列名与属性名不一致问题）

https://blog.csdn.net/qg_54351538/article/details/127616366

Mybatis中接口和对应的mapper文件位置配置详解

<https://blog.csdn.net/fanfanzk1314/article/details/71480954/>

Mybatis--动态SQL

<https://mybatis.org/mybatis-3/zh/dynamic-sql.html>

https://blog.csdn.net/qg_52797170/article/details/123981274

超全 MyBatis 动态 SQL 详解！(看完 SQL 爽多了)

<https://zhuanlan.zhihu.com/p/360448887>

Mybatis使用注解开发

https://blog.csdn.net/m0_46979453/article/details/122022095

四、MyBatisPlus

MyBatis-Plus 是一个 **MyBatis** 的增强工具，在 MyBatis 的基础上只做增强不做改变，为简化开发、提高效率而生。

具体使用，参考：

MyBatis-Plus官网：<https://baomidou.com/>

MyBatis-Plus详解

https://blog.csdn.net/weixin_66564094/article/details/126696648

MyBatis Plus详细教程

https://blog.csdn.net/m0_46313726/article/details/124187527

五、SpringBoot

SpringBoot是一个Java Web的开发框架，和SpringMVC类似。对比其他Java Web 框架，它能简化开发，约定大于配置，能迅速的开发Web应用。

SpringBoot是spring家族中的一个全新框架,用来简化spring程序的创建和开发过程。

Spring MVC是一个Web框架，用于构建Web应用程序；而 Spring Boot是一个依赖于Spring 的工具，用于简化Spring应用程序的配置和部署过程。Spring Boot本质上不是用来替代Spring MVC的，而是用来简化Spring应用程序，包括Web 应用程序的搭建和开发流程。

官方文档: <https://spring.io/projects/spring-boot>

如何在idea中创建一个SpringBoot项目（超详细教学）

https://blog.csdn.net/weixin_51309915/article/details/123349773

SpringBoot详解

https://blog.csdn.net/qg_46524280/article/details/124145626

SpringBoot基础篇(快速入门+要点总结)

https://blog.csdn.net/qg_53679373/article/details/130909995

六、Spring Cloud：微服务开发

Spring Cloud是一系列框架的有序集合。它利用Spring Boot的开发便利性巧妙地简化了分布式系统基础设施的开发，如服务发现注册、配置中心、消息总线、负载均衡、断路器、数据监控等，都可以用Spring Boot的开发风格做到一键启动和部署。

官方文档：

<https://spring.io/projects/spring-cloud>

学习教程：

https://blog.csdn.net/weixin_43714592/article/details/104238217

https://blog.csdn.net/m0_74823715/article/details/146432112

https://blog.csdn.net/weixin_54020399/article/details/132602244

七、WebSocket

WebSocket是双向通信协议,模拟Socket协议，可以双向发送或接受信息；HTTP是单向的。

WebSocket 是 HTML5 开始提供的一种浏览器与服务器间进行全双工通讯的网络技术。

即时通讯技术-WebSocket入门

<https://zhuanlan.zhihu.com/p/656062885>

WebSocket详解

https://blog.csdn.net/qg_54773998/article/details/123863493

Springboot整合Websocket

https://blog.csdn.net/qg_45443475/article/details/127442368

Springboot整合WebSocket

<https://blog.csdn.net/ZhangXS9722/article/details/127872663>

<https://www.cnblogs.com/jonil/p/17662742.html>

WebSocket在线测试

<http://www.websocket-test.com/>

八、团队协作开发项目

git

Git（读音为/git/）是一个开源的分布式版本控制系统，可以有效、高速地处理从很小到非常大的项目**版本管理**。

Git的功能特性：

从一般开发者的角度来看，git有以下功能：

- 1、从服务器上克隆完整的Git仓库（包括代码和版本信息）到单机上。
- 2、在自己的机器上根据不同的开发目的，创建分支，修改代码。
- 3、在单机上自己创建的分支上提交代码。
- 4、在单机上合并分支。
- 5、把服务器上最新版的代码fetch下来，然后跟自己的主分支合并。
- 6、生成补丁（patch），把补丁发送给主开发者。
- 7、看主开发者的反馈，如果主开发者发现两个一般开发者之间有冲突（他们之间可以合作解决的冲突），就会要求他们先解决冲突，然后再由其中一个人提交。如果主开发者可以自己解决，或者没有冲突，就通过。
- 8、一般开发者之间解决冲突的方法，开发者之间可以使用pull 命令解决冲突，解决完冲突之后再向主开发者提交补丁。

从主开发者的角度（假设主开发者不用开发代码）看，git有以下功能：

- 1、查看邮件或者通过其它方式查看一般开发者的提交状态。
- 2、打上补丁，解决冲突（可以自己解决，也可以要求开发者之间解决以后再重新提交，如果是开源项目，还要决定哪些补丁有用，哪些不用）。
- 3、向公共服务器提交结果，然后通知所有开发人员。

gitee/github

gitee

Gitee 是开源中国社区2013年推出的**基于 Git**的代码托管服务，目前已经成为国内知名的**代码托管平台**，致力于为国内开发者提供优质稳定的托管服务。

官网：<https://gitee.com/>

主要功能

Gitee 除了提供最基础的 Git 代码托管之外，还提供代码在线查看、历史版本查看、Fork、Pull Request、打包下载任意版本、Issue、Wiki、保护分支、代码质量检测、PaaS项目演示等方便管理、开发、协作、共享的功能。

具体使用，参考以下文献：

https://blog.csdn.net/m0_54722399/article/details/127133200

https://blog.csdn.net/kid_0_kid/article/details/121675420

https://blog.csdn.net/qg_52552691/article/details/123673482

github

GitHub是一个面向开源及私有软件项目的托管平台，因为只支持Git作为唯一的版本库格式进行托管，故名GitHub。

官网：<https://github.com/>

具体使用，参考以下文献：

<https://blog.csdn.net/jdsaiasodh/article/details/124667680>

<https://www.cnblogs.com/tuuli/p/16989916.html>

九、JavaScript框架

常见JS框架如下：

Vue.js

适用于从简单页面到复杂的单页应用程序，特别是需要快速开发和跨平台的场景。

<https://cn.vuejs.org/>

React

适用于需要高度交互性和动态内容更新的应用，如社交媒体网络应用程序、电子商务网络应用程序。

<https://zh-hans.react.dev/>

Angular

适合构建大型、复杂的企业级应用程序，如视频流应用程序、电子商务应用程序。

<https://www.angular.cn/>

<https://v4.angular.cn/>

十、UI框架

Web前端常见UI框架：

Element UI

基于Vue的PC端组件库，常用于后台管理系统。

<https://element.eleme.io/#/zh-CN>

Ant Design

蚂蚁集团的企业级React组件库，提供丰富的设计规范和组件。

<https://ant.design/index-cn/>

Bootstrap

经典响应式框架，支持HTML/CSS/JS开发，适合快速构建移动优先的Web项目。

<https://getbootstrap.com/>

十一、开源项目

推荐学习以下开源项目：

https://gitee.com/y_project/RuoYi

https://gitee.com/y_project/RuoYi-Cloud

<https://gitee.com/log4j/pig>

<https://gitee.com/zhongpengrui/jeecg-boot>

可以去gitee和github上去找更多的开源项目学习。

十二、AI应用编程

准备工作：去大模型平台申请API KEY，如deepseek，阿里千问，MiniMax，智谱，豆包大模型-火山引擎等，获取相应的base_url。

（一）基于Java的AI应用编程

1、Spring AI

Spring AI是Spring官方推出的AI应用开发框架，专为Java和Spring生态设计，让你能像调用数据库一样轻松集成AI大模型能力。

官网：<http://spring.io/projects/spring-ai>

文档：<http://docs.spring.io/spring-ai/reference>

示例：<https://github.com/spring-projects/spring-ai>

Spring AI 2.0技术栈要求

- Java 21（强制要求，不再支持 Java 17）

- Spring Boot 4.0.1
- Spring Framework 7.0

重要提示：由于底层架构重构，Spring AI 2.x 与 Spring Boot 3.x 不兼容，不可混合使用。

参考文献：

Spring AI 2.x 全面指南：架构升级、高效的工具调用、多模型生态与实战示例

<https://www.cnblogs.com/yangykaifa/p/19745500>

2026年Java AI开发实战：Spring AI完全指南

<https://zhuanlan.zhihu.com/p/2026364563268879288>

真正的企业级 Spring AI 2.0 实战指南（非常详细），高并发RAG落地看这篇就够了！

<https://mcp.csdn.net/6a2e308e10ee7a33f27cc8fc.html>

2、LangChain4j

LangChain4j是专门为Java开发者打造的AI应用开发框架，就是 Java 版本的 LangChain。

它是一个开源的 Java 框架，旨在简化大语言模型（LLM）在 Java 应用程序中的集成。它提供了一套统一的 API 和工具库，帮你屏蔽了底层不同模型（如 OpenAI、Azure、Qwen）的接口差异，让你能像调用普通 Java 服务一样调用 AI 能力。

- LangChain4j官方文档
- <https://docs.langchain4j.dev/>
- LangChain4j官方网站
- <https://github.com/langchain4j/langchain4j>
- LangChain4j示例项目
- <https://github.com/langchain4j/langchain4j-examples>
- LangChain4j 中文文档
- <https://langchain4j.cn/>

参考文献：

LangChain4j 从 0 到 1 实战教程

<https://aicoding.csdn.net/6a23e50310ee7a33f278a827.html>

LangChain4j 万字教程从零到一：Java开发者的大模型入门完全指南

<https://aicoding.csdn.net/6a238fc3662f9a54cb7a639f.html>

LangChain4j 实战指南：用 Java 轻松构建 AI 应用

<https://www.cnblogs.com/linlf03/p/20242017>

SpringBoot集成LangChain4j

<https://www.cnblogs.com/ludangxin/p/18937440>

<https://adg.csdn.net/696f2f6a437a6b403369a06a.html>

<https://juejin.cn/post/7510240927275221018>

(二) 基于Python的AI应用编程

1、LangChain

LangChain 是一个用于构建大语言模型（LLM）应用的开源编程框架，于2022年10月创建。目前LangChain已经在python和nodejs上实现了两种语言版本。

LangChain 的官方网站及相关核心资源链接如下：

 官方网站与文档

LangChain 官方主页: <https://www.langchain.com/>

Python 官方文档: <https://python.langchain.com/>

LangChain 中文文档: <https://python.langchain.cn/>

 其他核心资源

GitHub 仓库: <https://github.com/langchain-ai/langchain>

LangSmith (调试与监控平台): <https://smith.langchain.com/>

LangChain Hub (模板中心): <https://hub.langchain.com/>

API 参考文档: <https://api.python.langchain.com/>

参考文献：

LangChain 1.X 完全指南：

本书是一本系统学习 LangChain 1.X 版本的实战教程。

<https://www.learngraph.online/LangChain%201.X/README.html>

7步速成LangChain核心技术：Chain、Memory、RAG、Agent与LangGraph！

https://blog.csdn.net/2401_84204207/article/details/155104705

LangChain1.0速通指南：

LangChain1.0速通指南（一）——LangChain1.0核心升级

<https://zhuanlan.zhihu.com/p/1968427472388335014>

LangChain1.0速通指南（二）——LangChain1.0 create_agent api 基础知识

<https://zhuanlan.zhihu.com/p/1969383902624842213>

LangChain1.0速通指南（三）——LangChain1.0 create_agent api 高阶功能

<https://zhuanlan.zhihu.com/p/1970460972620679090>

深入浅出LangChain AI Agent智能体开发教程：

深入浅出LangChain AI Agent智能体开发教程（一）——认识LangChain&LangGraph

<https://zhuanlan.zhihu.com/p/1928437328675841858>

深入浅出LangChain AI Agent智能体开发教程（二）——LangChain接入大模型

<https://zhuanlan.zhihu.com/p/1929163379139929321>

深入浅出LangChain AI Agent智能体开发教程（三）——LangChain核心概念“链”

<https://zhuanlan.zhihu.com/p/1930977567562761991>

深入浅出LangChain AI Agent智能体开发教程（四）—LangChain记忆存储与多轮对话机器人搭建

<https://zhuanlan.zhihu.com/p/1933139780415258972>

深入浅出LangChain AI Agent智能体开发教程（五）—LangChain接入工具基本流程

<https://zhuanlan.zhihu.com/p/1933872247946352021>

深入浅出LangChain AI Agent智能体开发教程（六）—LangChain Agent API快速搭建智能体

<https://zhuanlan.zhihu.com/p/1934766479577940645> API版本旧了

注：创建智能体API已升级为：create_agent；原有的API create_tool_calling_agent过时了。

LangChain1.0实战：

LangChain1.0实战之多模态RAG系统（一）——多模态RAG系统核心架构及智能问答功能开发

<https://zhuanlan.zhihu.com/p/1974082873075201117>

LangChain1.0实战之多模态RAG系统（二）——多模态RAG系统图片分析与语音转写功能实现

<https://zhuanlan.zhihu.com/p/1976440517077275957>

LangChain1.0实战之多模态RAG系统（三）——多模态RAG系统PDF解析功能实现

<https://zhuanlan.zhihu.com/p/1978757223221065486>

LangChain1.0实战之多模态RAG系统（四）——Trae Solo搭建部署多模态RAG前端(附AI编程实践指南)

<https://zhuanlan.zhihu.com/p/1981331186551898643>

LangChain v1.x Models 完全指南：从入门到精通

<https://juejin.cn/post/7600622813457661967>

2、LangGraph

LangGraph 是基于 LangChain 构建的一种 状态机驱动的智能体编排框架，它将 Agent 系统建模设计为(有向图)流程图，每个节点为一个计算步骤，每条边()表示状态转移路径，适合处理复杂流程或多Agent 协作系统。

参考文献：

《LangGraph 1.0 完全指南》发布

https://www.luochang.ink/posts/dive_into_langgraph/

电子书：<https://www.luochang.ink/dive-into-langgraph/>

GitHub 仓库：<https://github.com/luochang212/dive-into-langgraph>

LangChain 1.0与LangGraph 1.0实战：从智能体构建到RAG与多智能体工作流

https://blog.csdn.net/weixin_30588675/article/details/95739022

LangChain&LangGraph 系列文章

https://blog.csdn.net/hjxu2016/category_12914292.html

【LangGraph入门 1】第一个LangGraph-HelloWorld

<https://blog.csdn.net/hjxu2016/article/details/160504182>

【LangGraph入门 2】串行控制和分支控制

<https://blog.csdn.net/hjxu2016/article/details/160594285>

【LangGraph入门 3】精细控制之图的运行时配置和map-reduce

<https://blog.csdn.net/hjxu2016/article/details/160597956>

【LangGraph入门 4】持久化与记忆

<https://blog.csdn.net/hjxu2016/article/details/160657549>

LangGraph1.0速通指南：

LangGraph1.0速通指南（一）—— LangGraph1.0 核心概念、点、边

<https://zhuanlan.zhihu.com/p/1985043690805276752>

LangGraph1.0速通指南（二）—— LangGraph1.0 条件边、记忆、人在回路

<https://zhuanlan.zhihu.com/p/1986064086958628943>

LangGraph1.0速通指南（三）—— LangGraph1.0 自动邮件处理智能体实战

<https://zhuanlan.zhihu.com/p/1987075481045063532>

智能体开发设计模式：

LangGraph智能体开发设计模式（一）——提示链模式、路由模式、并行化模式

<https://zhuanlan.zhihu.com/p/1987924873184556398>

LangGraph智能体开发设计模式（二）——协调器-工作者模式、评估器-优化器模式

<https://zhuanlan.zhihu.com/p/1989261268515852365>

LangGraph智能体开发设计模式（三）——LangGraph多智能体设计模式：主管架构与分层架构

<https://zhuanlan.zhihu.com/p/1991500023242974174>

LangGraph智能体开发设计模式（四）——LangGraph多智能体设计模式：网络架构

<https://zhuanlan.zhihu.com/p/1993101900971803000>

（三）AI编程工具

1、国产AI编程工具

腾讯CodeBuddy、字节Trae、阿里Qoder/通义灵码、百度Comate、智谱ZCode等。

2、国外AI编程工具

Cursor、Claude Code、CodeX、GitHub Copilot等。